

# Tensor Parallelism with CUDA - Matrix Multiplication

Tensor Parallelism from Scratch



DENNIS KENNETZ

FEB 21, 2025



12



2



Share

## Intro:

In the last post, I spent some words building up to what a **tensor** is and describing why they are useful in applications that benefit from high dimensionality, like machine learning. I then extended to the benefits of **tensor parallelism** - or splitting tensor operations over multiple GPUs. If you missed it and could use a laymen's refresher on tensors, check it out [here](#) before diving into this article because we buildin' baby.

For those just jumping into the series, I came to machine learning from a bioinformatics algorithms background and don't have a ton of experience with the mechanics underlying the tools I use every day. However, I do understand how GPUs work, and how to write CUDA code to utilize the GPUs to do work, as I spent 99% of my dev work in my previous role writing HPC algorithms in CUDA / C++. Now, a lot of this is abstracted away from me by some amazing Python libraries like PyTorch, TensorFlow, and JAX so I still don't really get to interact with the "meat" of what runs these amazing ML algorithms - and that is what I'm setting out to explore.

In this article, we will break down matrix multiplication “on paper” before implementing a simple matrix multiplication (matmul) in CUDA. **Note: I do not plan on diving into optimizations of matmul, as that is one of *the most well-studied optimizations in computing*.** Instead, I just want to drill into concepts. Specifically - **tensor parallelism** - and matmul is a great representative structure to do just that. I’m going to do my best to make this content approachable for people without a math background, but it’s important to note that the very term “tensor” is a mathematical concept. This article gets a bit deeper than its predecessor since I actually *will* perform tensor (matrix) operations here. Just hang in there with me.

## Scary Math Notations like I’m 15

Coming fresh off the fact that we know a **tensor** is a higher dimensional abstraction for a representation of numbers, and we know a **matrix** is a tensor of **rank-2** (seriously, last warning, [read the last article if you don't know what I'm talking about](#)), we can use this information to tell a story to represent our example tensor.

Tensor parallelism (TP) refers to partitioning tensor operations across multiple computational units (e.g., GPUs in this case) to enable parallel computation. Mathematically, this means splitting large tensor computations into smaller subcomputations that can be executed concurrently.

Consider a large matrix multiplication:

$$C=A \times B$$

Where (+1 for LaTeX - it’s been awhile):

$$A \in \mathbb{R}^{m \times k}$$

$$B \in \mathbb{R}^{k \times n}$$

$$C \in \mathbb{R}^{m \times n}$$

**Pause — Let's explain:**

$A \in \mathbb{R}^{m \times k}$  :

- “A” is the name of a matrix
- “ $\in$ ” means “is in” - “Dennis is in the house”
- “ $\mathbb{R}^{m \times k}$ ”: table (of real numbers denoted by the “R”) of m rows and k columns (m x k matrix)
- Finally, “A is in the table of real numbers with dimensions m x k.”

$B \in \mathbb{R}^{k \times n}$  :

- Same meaning as above, except the matrix B is in (k x n matrix)

$C \in \mathbb{R}^{m \times n}$  :

- Same as above, but this is the resultant matrix, so C is in (m x n matrix)

OK, so where did the k go when we look at C?

When you multiply A (m x k) by B (k x n), the k value is a dimension in each, and it must match, and they cancel each other out. If they do not match, the matrices cannot be multiplied and dimensions of one of the two matrices would need to be adjusted such that A did have k columns or B did have k rows.

However, given our starting example - the matrices do match, and the resulting matrix is C (m x n).

## Let's Peep an Example:

So we can understand *just how many operations* occur, and also *how easy these are to parallelize* we will actually walk through an example of matrix multiplication (on paper as they say).

Consider the following matrix where  $C = A \times B$  again where A and B are represented below:

$$A = \begin{bmatrix} 1 & 2 & 3 \\ 4 & 5 & 6 \end{bmatrix} \quad B = \begin{bmatrix} 100 & 200 \\ 300 & 400 \\ 500 & 600 \end{bmatrix}$$

And C is represented by variables that we will show calculations for

$$C = \begin{bmatrix} c_{11} & c_{12} \\ c_{21} & c_{22} \end{bmatrix}$$

How do we actually do the multiplication of these two matrices?

- multiply each item of row 1 of A by each item of column 1 of B and add them
- multiply each item of row 1 of A by each item of column 2 of B and add them
- multiply each item of row 2 of A by each item of column 1 of B and add them
- multiply each item of row 2 of A by each item of column 2 of B and add them

Let's do this by starting with  $c_{11}$ :

$$c_{11} = [1 \quad 2 \quad 3] \begin{bmatrix} 100 \\ 300 \\ 500 \end{bmatrix} = 1(100) + 2(300) + 3(500) = 100 + 600 + 1500 = 2200$$

Yay! We've found the first item in our result C, 2200.

$$c_{12} = [1 \quad 2 \quad 3] \begin{bmatrix} 200 \\ 400 \\ 600 \end{bmatrix} = 1(200) + 2(400) + 3(600) = 200 + 800 + 1800 = 2800$$

We've found our second result of C, 2800. Now moving to row 2.

$$c_{21} = [4 \quad 5 \quad 6] \begin{bmatrix} 100 \\ 300 \\ 500 \end{bmatrix} = 4(100) + 5(300) + 6(500) = 400 + 1500 + 3000 = 4900$$

Okay 4900, one to go:

$$c_{22} = [4 \quad 5 \quad 6] \begin{bmatrix} 200 \\ 400 \\ 600 \end{bmatrix} = 4(200) + 5(400) + 6(600) = 800 + 2000 + 3600 = 6400$$

And our final result is 6400 for C, yielding the final matrix:

$$c_{22} = \begin{bmatrix} 2200 & 2800 \\ 4900 & 6400 \end{bmatrix}$$

Thanks for sticking with me.

**Notice Anything?**

As we went through the above operations “on paper”, did you notice anything? This was a small array, and I see 4 completely distinct calculations. Namely,  $c_{11}$  had no dependence on  $c_{12}$ . Neither had dependence on  $c_{21}$ , and none on  $c_{22}$ . What should that tell you?

**We can do these in parallel.**

Herein lies the heart of our tensor parallelism! While our example here was trivial, imagine if our matrices were of dimensions  $[16384 \ 8192]$  and  $[8192 \ 8192]$ . To get a quick idea, we can calculate the total operations required for our matmul by considering the operations per element for each element  $C$ :

- $k$  multiplications
- $k - 1$  additions

Therefore, the total number of operations per element is:

- $2k - 1$

For our entire matrix, this becomes:

- total operations =  $(m \times n) \times (2k - 1)$

So, given our  $[2 \times 3]$  and  $[3 \times 2]$  matrices:

- $(2 \times 2) \times (2(3) - 1) = 4 \times (6 - 1) = 20$  total operations

Blow that up to  $[16384 \times 8192]$  and  $[8192 \times 8192]$ :

- $(16384 \times 8192) \times (2(8192) - 1) = 134217728 \times (16384 - 1) = 2,198,889,037,824$

Yep, that's nearly **2.2 Trillion operations**. Sure, some of you brainiacs may be thinking that some of these add + multiply operations can be fused at the hardware level, but it's still immense. Now we can appreciate the marvel of machines like the H100 which boast 51-67 teraFLOPs, or 51-67 trillion floating point operations per second (and we haven't even touched on reduced precision).

## Okay, so where's the code?

This article has dragged on a bit, and I feel like this exercise would be incomplete without first demonstrating our simple “on paper” matrix multiplication in CUDA before jumping into tensor parallelism - which we will visit next time.

The naive matrix multiplication kernel is simple enough:

```
#define M 2 // Rows of A, Rows of C
#define N 2 // Columns of B, Columns of C
#define K 3 // Columns of A, Rows of B

__global__ void matMulKernel(int *A, int *B, int *C, int m, int n, int k) {
    int row = blockIdx.y * blockDim.y + threadIdx.y;
    int col = blockIdx.x * blockDim.x + threadIdx.x;

    if (row < m && col < n) {
        int value = 0;
        for (int i = 0; i < k; i++) {
            value += A[row * k + i] * B[i * n + col];
        }
    }
}
```

```
        C[row * n + col] = value;
    }
}
```

If you are unfamiliar with CUDA and feel lost looking at the function, don't worry too much.

Some important things to note for those new to CUDA:

- `__global__` defines a kernel, which is a function that is launched from the host (CPU) with threads specified at launch
- `*A`, `*B`, `*C` are pointers to contiguous arrays in device (GPU) memory
- `blockIdx`, `blockDim`, and `threadIdx` are CUDA built-in variables which provide information about the structure of threads and blocks within a kernel
- These are not super relevant today, except to know that they are generally how we index into device arrays without carrying around extra information

The anatomy of the kernel:

- It is common to wrap calculations or things which index into a device array with a condition like the one seen above, as thread and block index may access illegal memory if you let them go past the bounds of which has been allocated - bad!
- The remainder is indexing properly into the array given the correct row, column, and indexing offsets. I will let you work out the math, and I've added a helpful `printf` in the kernel so you can inspect numbers.



Drumroll for final result...

```
row: 0, col: 0  
row: 0, col: 1  
row: 1, col: 0  
row: 1, col: 1  
Result Matrix C:  
2200 2800  
4900 6400
```

It matches our paper math! Nice job team.

## Final Remarks

If you made it here, great job. I'd love to jump right into tensor parallelism, but we need to reinforce important concepts along the way to make sure we *really* understand what is going on under the hood. After all, we are implementing it from scratch. Now that we understand matrix multiplication, and how it lends very well to **parallelism**, in the weeks to come I'll jump into different types of parallelism, some more complex constructs used in machine learning like MLP and attention, and go from there.

The full code from today's article exists in my `cuda_examples` repository below:

[https://github.com/drkennetz/cuda\\_examples/tree/main/TensorParallelFromScratch/01\\_simple\\_matmul\\_no\\_tp](https://github.com/drkennetz/cuda_examples/tree/main/TensorParallelFromScratch/01_simple_matmul_no_tp)

Thanks for sticking around, and if you'd like to follow the code above, don't forget to star and watch the repo.